🚀 Day 19: PySpark Window Functions & Handling Missing Data

21-Day Data Engineering Sprint • The Data Refinery Masterclass

Author: Prasad S Hiremath | Technical Curriculum & Hands-on Lab

> 💡 **Daily Motivation & Philosophy:**
>
> *"Focus starts with elimination, improves with concentration, and compounds with continuation."*
>
> *"I am not afraid of a person who knows 10,000 different kicks, I am afraid of a person who practiced a single kick 10,000 times."* — **Bruce Lee**
>
> *"In the same span of time, one individual builds results through deliberate action, while another remains idle, trapped in overthinking. Success is not defined by the time available, but by how purposefully it is used."*
>
> Welcome to Day 19! You have conquered groupings and joins. Today, we step into advanced analytics with Window Functions—calculating running totals and rankings without collapsing your rows—plus mastering production data cleaning for null values. Let's keep compounding our momentum!

🏢 The Real-World Scenario: The Running Total Collapse at "InstaDrop"

At **InstaDrop**, management wanted to track each customer's cumulative spending over time to identify high-value VIP milestones.

A junior data engineer tried using a standard `groupBy("customer_id").agg(sum("amount"))`:

- **The Problem:** `groupBy` collapsed the rows! It returned only one row per customer, erasing individual order dates, order IDs, and the chronological sequence of purchases.

How do Window Functions solve this? A **Window Function** performs calculations across a set of rows (a "window") related to the current row, **without collapsing the DataFrame**. Every original row is preserved while adding rich analytical insights like running totals, rankings, and moving averages!

📖 Part 1: Core Concepts Explained Simply

1️⃣ PySpark Window Functions (`Window`, `row_number`, `sum` over window)

Think of a Window function as a **sliding magnifying glass** inspecting a specific subset of rows. To define a window, you use three components:

1. **`partitionBy()`:** Splits data into groups (like `customer_id`).
2. **`orderBy()`:** Sorts rows within each partition chronologically.
3. **`rowsBetween()`** / **`rangeBetween()`** **(Optional):** Defines frame boundaries (e.g., from the beginning of the partition up to the current row for running totals).

```
from pyspark.sql.window import Window
from pyspark.sql.functions import col, sum, row_number

# Define window: Partition by customer, order by timestamp
windowSpec = Window.partitionBy("customer_id").orderBy("order_date")

# Calculate running cumulative spend per customer
running_total_df = df.withColumn("running_total", sum("amount").over(windowSpec))
```

2️⃣ Handling Missing Data (`.na.drop()` & `.na.fill()`)

Real-world data is messy and full of missing (`null`) values. PySpark provides a robust `.na` toolkit for cleaning datasets:

- **Drop rows with any nulls:** `df.na.drop()`
- **Drop rows only if specific columns are null:** `df.na.drop(subset=["amount", "customer_id"])`
- **Fill null values with defaults:** `df.na.fill({"amount": 0.0, "status": "UNKNOWN"})`

📋 WhatsApp Quick Reference Table

Operation	Python PySpark Syntax	Production Purpose
Define Window Partition	`Window.partitionBy("col")`	Splits data into isolated analytical groups.
Add Window Ordering	`.orderBy("timestamp")`	Establishes chronological sorting within each partition.
Running Cumulative Total	`sum("val").over(windowSpec)`	Computes running sums without collapsing rows.
Row Ranking	`row_number().over(window)`	Assigns sequential integer ranks within partitions.
Fill Null Values	`df.na.fill({"col": "val"})`	Replaces missing null entries with default fallbacks.
Drop Null Rows	`df.na.drop(subset=["id"])`	Removes records missing mandatory primary keys.

💻 Part 2: Beginner-Level Coding Challenge & Full Solution

Scenario: Clean raw order logs for InstaDrop containing null values, assign a chronological sequence number (`order_sequence`) to each customer's purchases, and compute their running cumulative spend.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType
from pyspark.sql.window import Window
from pyspark.sql.functions import col, sum, row_number

# 1. Initialize SparkSession
spark = SparkSession.builder     .appName("InstaDrop_Day19_Windows")     .master("local[*]")     .getOrCreate()

# 2. Raw dataset with null values
raw_data = [
    ("ORD-01", 101, 250.0, None),
    ("ORD-02", 101, None, "COMPLETED"),  # Null amount
    ("ORD-03", 101, 450.0, "COMPLETED"),
    ("ORD-04", 102, 1000.0, "COMPLETED"),
    (None,     102, 150.0, "CANCELLED")   # Null order_id
]

schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("customer_id", IntegerType(), True),
    StructField("amount", DoubleType(), True),
    StructField("status", StringType(), True)
])

df = spark.createDataFrame(raw_data, schema=schema)

# 3. Data Cleaning: Drop rows with missing order_id, fill missing amount with 0.0, fill status
cleaned_df = df.na.drop(subset=["order_id"])                .na.fill({"amount": 0.0, "status": "UNKNOWN"})

# 4. Define Window Specification
windowSpec = Window.partitionBy("customer_id").orderBy("order_id")

# 5. Apply Window Functions (Row Number & Cumulative Spend)
windowed_df = cleaned_df.withColumn("order_sequence", row_number().over(windowSpec))                            .withColumn("cumulative_spend", sum("amount").over(windowSpec))

print("--- InstaDrop Window Analytics & Cleaned Data ---")
windowed_df.show()

spark.stop()
```

Execution Output:

```
--- InstaDrop Window Analytics & Cleaned Data ---
+--------+----------+------+---------+--------------+----------------+
|order_id|customer_id|amount|   status|order_sequence|cumulative_spend|
+--------+----------+------+---------+--------------+----------------+
|  ORD-01|       101| 250.0|  UNKNOWN|             1|           250.0|
|  ORD-02|       101|   0.0|COMPLETED|             2|           250.0|
|  ORD-03|       101| 450.0|COMPLETED|             3|           700.0|
|  ORD-04|       102|1000.0|COMPLETED|             1|          1000.0|
+--------+----------+------+---------+--------------+----------------+
```

❓ Part 3: 5 Core Concept MCQs with Detailed Solutions

CORE QUESTION 1

What is the primary operational difference between a groupBy() aggregation and a Window function (.over(windowSpec)) in PySpark?

A) Window functions delete all null values, whereas groupBy retains them.
B) GroupBy collapses rows into a single summary row per group, whereas Window functions compute metrics across related rows while preserving every individual row.
C) Window functions run only on the driver node, whereas groupBy runs on worker nodes.
D) GroupBy is exclusively used for machine learning models.

🔑 **Correct Answer: B**
Detailed Reasoning: Standard aggregations (`groupBy`) compress multiple rows into a single aggregated row. Window functions allow you to calculate running totals, ranks, or moving averages while keeping granular row-level details intact.

CORE QUESTION 2

When defining a Window specification (Window.partitionBy("customer_id")), what does partitionBy achieve?

A) It splits the physical file storage across different cloud storage buckets.
B) It divides the DataFrame into independent groups (partitions) so that window calculations (like rankings or sums) are computed separately within each group.
C) It deletes duplicate customer records from disk.
D) It sorts the dataset in descending order.

🔑 **Correct Answer: B**
Detailed Reasoning: `partitionBy()` groups rows logically (e.g., grouping all transactions by each unique customer) so that window functions operate independently within each customer's isolated boundary.

CORE QUESTION 3

If you execute df.na.fill({"amount": 0.0, "status": "PENDING"}) on a PySpark DataFrame, what happens?

A) All rows containing any null value in the entire DataFrame are permanently deleted.
B) Null values in the amount column are replaced with 0.0, and null values in the status column are replaced with "PENDING".
C) The DataFrame schema is converted entirely to string format.
D) Spark throws a NullPointerException error.

🔑 **Correct Answer: B**
Detailed Reasoning: `.na.fill(value_dict)` allows data engineers to specify column-specific default fallback values to replace `null` entries cleanly without affecting other columns.

CORE QUESTION 4

What does the row_number().over(windowSpec) function return when evaluated over a sorted window?

A) The total count of rows in the entire dataset.
B) A sequential integer rank (1, 2, 3...) assigned to each row within its specific partition based on the window's sorting order.
C) The memory address of the worker node executing the task.
D) The percentage contribution of the row's amount.

🔑 **Correct Answer: B**
Detailed Reasoning: `row_number()` assigns a unique, sequential integer starting at 1 to each row within its partition, respecting the specified `orderBy()` clause.

CORE QUESTION 5

Why is passing a subset parameter (e.g., df.na.drop(subset=["order_id"])) recommended over bare df.na.drop() in production pipelines?

A) Bare df.na.drop() deletes an entire row if any column has a null, which can needlessly discard valid records if an optional column (like middle name or discount) is missing.
B) subset forces Spark to execute queries on local CPU memory.
C) Bare df.na.drop() causes syntax errors in Python 3.
D) subset automatically encrypts sensitive columns.

🔑 **Correct Answer: A**
Detailed Reasoning: Unqualified `.na.drop()` removes any row where *any* column contains a null. Specifying `subset` ensures you only drop rows missing critical mandatory primary keys or essential operational columns.

🌟 Part 4: 3 Beginner-Friendly Bonus Q&As

BONUS QUESTION 6

What is the difference between row_number(), rank(), and dense_rank()?

💡 **Detailed Answer & Reasoning:**

- `row_number()` assigns unique sequential integers (1, 2, 3, 4) regardless of ties.
- `rank()` assigns the same rank to tied values and leaves gaps in the ranking sequence (e.g., 1, 2, 2, 4).
- `dense_rank()` assigns the same rank to ties but leaves **no gaps** in the sequence (e.g., 1, 2, 2, 3).

BONUS QUESTION 7

Can you perform moving averages using PySpark Window frames?

💡 **Detailed Answer & Reasoning:**
Yes! By defining frame boundaries using `.rowsBetween()`. For example, `Window.partitionBy("item").orderBy("date").rowsBetween(-2, 0)` tells Spark to calculate the window aggregation using the current row and the two preceding rows, perfect for moving averages and rolling financial metrics.

BONUS QUESTION 8

Do Window functions trigger a shuffle in Spark?

💡 **Detailed Answer & Reasoning:**
Yes. Because `partitionBy()` requires all rows sharing the same partition key (e.g., `customer_id`) to be gathered onto the same worker node, Spark triggers a **Shuffle** across the cluster. If your partitioning key has high data skew (e.g., 90% of null keys), it can cause a severe performance bottleneck.